Basketball RAG

NLP Course Project

Saad Imam 27079

Github Repository Link
App Link (Hosted on Hugging face)

1 Business Problem

Basketball is a sport whose technical depth is underestimated. At the casual level, fans can follow a game but struggle to understand *why* a referee call was made, how a specific play exploits a defensive scheme, or what the structural difference is between a pick-and-roll and a pick-and-pop. At the coaching level, communicating precise rules interpretations to players, or explaining the principles behind a tactical set, requires access to dense, technically heavy rulebooks and strategy literature.

The core problem this system addresses is the **accessibility gap** between institutional basketball knowledge (official rulebooks, tactical encyclopedias) and the people who need that knowledge most. Existing tools either expose raw PDFs with no semantic search, or require a domain expert to answer every question.

The inspiration for the system's framing came from *Mind the Game* and the older *Old Man and the Three* podcasts hosted by JJ Redick. As a basketball fan who started watching the game in 2022, I had issues with understanding what was happening on the court tactically. That is when I came across JJ Redick, a 15 year NBA veteran, who had 15-20 mins segments in his podcast explaining specific plays in clear, understandable language. This system aims to replicate that quality: answering questions from a casual fan asking "what is a defensive three-second violation?" and a coach asking "how does a Princeton offense attack a zone defence?" with equal precision.

The system deliberately excludes statistical lookup (e.g., "who led the league in three-point percentage in 2021?"), as this requires structured databases and is a qualitatively different problem from rules and strategy retrieval.

1.1 Expected Challenges

Several challenges were anticipated at the outset:

Corpus differences: The two data layers (official rulebooks in PDF format and a scraped tactical encyclopedia) differ significantly in structure, tone, vocabulary, and document length. A unified retrieval strategy must serve both equally well.

Technical query specificity: Queries about basketball rules often contain precise identifiers — rule numbers, section titles, league names — that a purely semantic dense retriever may not surface correctly. Hybrid retrieval was therefore identified as a core requirement from the beginning.

Faithfulness vs. fluency trade-off: A language model with parametric basketball knowledge may confidently answer a question without grounding its answer in the retrieved context. The system must enforce strict grounding to be trustworthy, at some cost to response fluency.

2 Platform Details

The system is deployed as a **Docker Space on Hugging Face Spaces**, exposing a **FastAPI** backend on port 7860 with a coded single-page front-end. Source code is hosted on GitHub, with a GitHub Actions workflow (`.github/workflows/sync-to-hf.yml`) that automatically pushes to the Hugging Face Space repository on every push to the `main` branch, keeping the two repositories in sync without manual input.

Some Key Notes:

1. **Hugging Face Spaces (Docker SDK):** Provides free-tier hosted inference without requiring a dedicated server. The Docker SDK was chosen over Gradio or Streamlit to avoid the constraints those frameworks impose on UI design; it was a personal goal to have a customized front-end rather than looking generic.

2. **Local Machine vs. Kaggle:** For the scraping and chunking tasks, the local machine was used as it was not a very GPU intensive task. However, BGE-M3 embedding of the full corpus was computationally infeasible on a local CPU within a reasonable time window. A Kaggle notebook with dual T4 GPUs was used to run the embedding and upsert pipeline, with embeddings saved to a local checkpoint file before upserting to Pinecone, so that a failed upsert does not require re-running the 30–60 minute embedding step.

3. The corpus files (raw PDFs and scraped JSON) are **not** included in the Hugging Face repository; all vectors and metadata reside in Pinecone and are queried at inference time. Model artefacts (BGE-M3, CrossEncoder) are downloaded lazily on the first request to avoid blocking the `uvicorn` server from binding to port 7860 during startup — an issue that had previously caused the Space to timeout and loop indefinitely.

3 Corpus Details

The corpus is organised into two layers:

3.1 Layer 1

- **NBA Official Playing Rules (2025–26)** This document covers all NBA rules across numbered rules and roman-numeral sections (e.g., Rule 10, Section XIII — Traveling), used for officiating games.
- **FIBA Official Basketball Rules 2024 (v10a).** Retrieved from the FIBA official website. Covers international rules, including differences in shot clock duration, foul limits, and game structure relative to the NBA.

Several page ranges were excluded from processing because they carry no retrieval value: for the NBA rulebook, pages 1–8 (the rules index and court diagram) and pages 72–76 (referee hand signals, which are image-based); for the FIBA rulebook, the front matter and appendix pages containing diagrams and administrative content. These exclusions were implemented by checking each chunk's page structure; more on this later.

Diagrams (X-and-O illustrations) present in both rulebooks were intentionally excluded. Current vision-language models do not reliably interpret the symbolic notation used in basketball diagrams, and including OCR artefacts from these pages would introduce noise rather than signal. Was also out of the scope of the project.

3.2 Layer 2

Layer 2 was scraped from `https://hoopstudent.com`, a basketball strategy and glossary website with three main category pages: *Basketball Glossary*, *Offense*, and *Defense*. Each category page lists terms with a 3–4 line concise definition and a link to a dedicated detail page. The scraper (`scraper.py`) performs the following steps for each term:

1. Parses the category hub page to extract the term name, concise definition, and detail URL.
2. Fetches the detail page and parses its content into structured sections, using heading elements (`h2`/`h3`) as section boundaries.
3. Structures the output as a JSON document with fields: `doc_id`, `term`, `category`, `source_url`, `layer`, `concise_definition`, and `sections[]`.

The decision to use JSON rather than plain text was made deliberately: structured metadata storage is essential for filtering and re-ranking during retrieval. With flat text files, information such as the source layer, term name, and category is either embedded in the text or lost entirely. With JSON, this information is preserved separately and stored alongside the vector in Pinecone.

The corpus does not include raw statistics, CSV box scores, or structured player records. The system is scoped to rules, definitions, and strategy — a deliberate framing that keeps retrieval tractable and the answer quality verifiable.

The directory structure of the processed corpus is as follows:

```
corpus/
  raw/
    Official-2025-26-NBA-Playing-Rules.pdf
    fiba-official-rules-2024-v10a.pdf
    layer2_hoopstudent.json
  processed/
    layer1_rulebook_chunks.json
    layer2_chunks.json
  embeddings_checkpoint.json
```

4 Implementation Journey

Below is a detailed account of the full implementation journey for the first iteration of the system. Modifications made specifically for the ablation study are discussed in the following chapter.

4.1 Pre-processing

Pre-processing differs substantially between the two layers due to their source formats.

Layer 1: Rulebook PDF Processing

PDF processing for Layer 1 is handled by **Docling** (IBM Research). The choice of Docling over alternatives such as `pdfminer` or `PyMuPDF` was deliberate. Traditional PDF extraction libraries produce a flat text stream — headers, body content, and page numbers are interleaved without structural distinction. For a rulebook organised hierarchically into Rules and Sections, this means the extractor has no reliable way to identify where one rule ends and another begins without fragile regular expressions tuned specifically to that document's formatting.

Docling applies computer vision models to understand document layout, producing a **DoclingDocument** object with heading nodes already identified and placed in their correct hierarchical positions. This makes downstream processing significantly cleaner: the heading path for any chunk of text (e.g., *"RULE NO. 10 — Violations and Penalties → Section XIII — Traveling"*) is accessible directly from the chunk's metadata without any text parsing.

The Docling pipeline was configured to disable OCR and table structure analysis, since the rulebook text is native PDF text and no tables require extraction.

Heading Parsing

After Docling chunks the document, each chunk's heading path is parsed by a custom `parse_heading_path()` function using regular expressions designed to handle the formatting conventions of both the NBA and FIBA rulebooks. The function extracts:

- `rule_number`: e.g., *Rule 10*
- `rule_title`: e.g., *Violations And Penalties*
- `section`: e.g., *Section XIII*
- `section_title`: e.g., *Traveling*

- `chunk_type`: one of *rule_section*, *definitions*, *comments*, or *general*

This structured metadata is stored alongside each vector in Pinecone and is used by the generator to construct informative source labels in the UI.

Noise Filtering

A `is_noise_chunk()` filter removes chunks that have no retrieval value: chunks shorter than 80 characters, chunks consisting entirely of digits and dashes (page number artefacts), index entry patterns ("Rule No. X ... Page Y"), and chunks containing dense sequences of dot leaders (table-of-contents entries). This filter operates after chunking, on the raw text of each candidate chunk.

Context Prefix

Each chunk's text is prepended with a natural-language context prefix constructed from its heading metadata:

```
"NBA Rulebook | Rule 10 (Violations And Penalties),
 Section XIII (Traveling):\n\n{chunk_text}"
```

This prefix serves an important retrieval function: without it, a chunk about traveling merely reads "A player who receives the ball while standing still may pivot...". There is no explicit indication that the text is from the NBA rulebook, from Rule 10, or even about basketball. With the prefix, BM25's sparse retrieval leg can match on "Rule 10", "Traveling", "NBA", and "Violations" as explicit keyword signals, substantially improving hybrid search precision for rule-specific queries.

Layer 2: HoopStudent JSON Pre-processing

Layer 2 pre-processing (`preprocess_layer2.py`) converts the scraped JSON documents into the standardised chunk format expected by the embedding and indexing pipeline. Each term document produces two types of chunks:

1. **Definition chunk.** A single short chunk constructed from the `concise_definition` field, formatted as `''{term}: {definition}''`. This chunk is optimised for exact-match queries ("what is a pick and roll?") because the term name and its definition are co-located in a small, dense text window, improving both BM25 keyword matching and dense embedding precision.

2. **Section chunks.** One chunk per section from the detail page, formatted with a context prefix: `''{term} | {section_heading}:\n\n{section_text}''`. These chunks carry the detailed explanatory content and handle queries that require more than a one-line answer (e.g., "how does a floppy play work against a switching defence?").

Sections with headings in a defined skip-list ("Table of Contents", "Introduction", navigational boilerplate) and sections shorter than 80 characters are discarded.

4.2 Chunking

Layer 1: HybridChunker

For the rulebook PDFs, chunking is performed by Docling's **HybridChunker**, which performs Document Specific Chunking. The alternative, `HierarchicalChunker`, splits strictly at heading boundaries regardless of section size — a two-sentence section such as "Section V — Free Throw" becomes a standalone chunk too short to embed meaningfully.

`HybridChunker` applies a smarter strategy: it merges short adjacent sections that share the same parent heading, splits long sections at sentence boundaries when they exceed the token limit, and preserves the heading hierarchy throughout. For a corpus where section lengths range from two sentences to three pages, this results in a significantly more uniform chunk size distribution.

The token limit was set to **500 tokens** using the BGE-M3 tokenizer (`BAAI/bge-m3`). BGE-M3 supports a maximum sequence length of 8192 tokens, but shorter chunks embed more precisely — a chunk that spans two unrelated sections risks producing an embedding that represents neither adequately. Critically, the **same tokenizer** must be used for chunking as for embedding, to ensure token counts are accurate.

Layer 2: Fixed Schema Chunking

Layer 2 does not use a sliding window or a general-purpose chunker. The scraped documents have a natural, pre-existing structure (concise definition + discrete sections), so the chunking strategy follows that structure directly. This avoids the risk of a fixed-window chunker splitting a definition mid-sentence or merging two unrelated sections.

4.3 Embedding

Model Selection: BGE-M3

The embedding model is **BGE-M3** (`BAAI/bge-m3`). The original plan involved separate dense and sparse encoders: `all-MiniLM-L6-v2` for dense embeddings and Pinecone's `BM25Encoder` for sparse. This was replaced by BGE-M3 for the following reasons:

1. BGE embedding models are very popular for their quality as well as effeciency.
2. BGE-M3 generates both a 1024-dimensional dense vector and a learned sparse vector (lexical weights) in a **single forward pass**, with no separate encoder or vocabulary

fitting required. The sparse representation is qualitatively superior to classical BM25: rather than frequency-based IDF weights over a fixed vocabulary, BGE-M3's lexical weights are learned from training data and reflect the actual semantic importance of tokens in context. This produces better keyword matching, particularly for domain-specific terminology such as basketball rule identifiers.

Sparse Vector Format

BGE-M3's `lexical_weights` output is a dictionary mapping token identifiers to importance scores. This is converted to Pinecone's required sparse format ({`indices: [...], values: [...]`}) with two filtering steps: weights below 0.01 are discarded as noise, and the result is capped at the top 1000 tokens by weight. The 1000-token cap is a practical limit imposed by Pinecone's sparse vector constraints, not related to the 1024-dimensional dense vector.

Pinecone Index Architecture

A single hybrid Pinecone index is used with the following configuration:

- **Dimension:** 1024 (BGE-M3 dense output)
- **Metric:** `dotproduct` — mandatory for hybrid sparse-dense search in Pinecone

Each Pinecone record stores the dense vector, the sparse vector, and a metadata dict containing all fields from the chunk's `metadata` key, including the chunk text itself (stored under the `text` key for retrieval-time access without a separate document store).

Due to the scale of the embedding operation (several hundred chunks, BGE-M3's computational requirements), the embedding step was run on a Kaggle notebook with dual T4 GPUs, as mentioned above. Due to network failures during Pinecone upsert, re-running the full embedding pass was required. To avoid this, checkpointing was used.

4.4 Retrieval

The retrieval pipeline (`retriever.py`) implements a four-stage process designed to balance recall, diversity, and precision:

1. **Query Encoding.** The query is encoded with BGE-M3 (same model used during indexing) to produce a 1024-dim dense vector and a learned sparse vector in one pass.
2. **Dual Pinecone Query:** Two separate Pinecone queries are issued against the same index: a pure dense query (`alpha = 1.0`) retrieving the top 20 results by semantic similarity, and a pure sparse query (`alpha = 0.0`) retrieving the top 20 results by

lexical weight overlap. These were run as separate queries, since we need two lists, a requirement for Reciprocal Rank Fusion.

3. **Reciprocal Rank Fusion (RRF):** The two ranked lists are fused using RRF with smoothing constant $k = 60$.

4. **CrossEncoder Re-ranking:** The top 10 candidates from RRF are passed to a CrossEncoder (`cross-encoder/ms-marco-MiniLM-L-6-v2`) for precision re-ranking. The CrossEncoder jointly encodes the query and each candidate as a single input, producing a relevance score. Applying it only to the small post-RRF candidate set (rather than the full corpus) keeps latency acceptable. The final top 5 candidates by CrossEncoder score are returned.

5. **Singleton loading:** Both BGE-M3 and the CrossEncoder are loaded once at application startup via singleton instances and reused across all requests.

A note on MMR. Maximal Marginal Relevance (MMR) was implemented in the pipeline but is disabled in the current iteration (the top-10 RRF results are passed directly to the CrossEncoder without diversity re-selection). MMR requires re-embedding the candidate texts with BGE-M3 to compute inter-document cosine similarity, which is fast at $\leq$15 documents but adds latency. Its effect relative to the CrossEncoder will be examined in the ablation study.

4.5 Generation

Model Selection Journey

Selecting a reliable, free generation backend proved to be one of the more operationally challenging aspects of the project. The intended model was `mistralai/Mistral-7B-Instruct-v0.3` via the HuggingFace Inference API. The journey through providers is summarised below:

- **HuggingFace Inference API (hf-inference):** No longer supports large LLM chat completion at the free tier.
- **Novita:** Returned 404 for Mistral.
- **Cerebras:** The older `huggingface-hub` library used incompatible chat completion syntax; resolved by switching to `.chat.completions.create()`, but the provider itself had reliability issues.
- **SambaNova:** Functional but drained the monthly HuggingFace credit allocation.
- **OpenRouter + `google/gemma-3-12b-it:free`:** Permanently free, OpenAI API-compatible (requiring no library changes beyond `base_url`), and backed by multiple providers for uptime. However, it was limited to 50 requests per day, making testing difficult.

- **Groq** + `llama-3.1-8b-instant`: Groq proved to be the most friendly API, with flexible rate limits applied per minute. Llama3.1-8B was also a good model according to online research for RAG based applications. This was finalized for the first implementation.

Prompt Design

The prompt enforces three faithfulness behaviours explicitly:

1. **Context-only answering.** The model is instructed to answer *only* from the numbered `[Source N]` blocks in the prompt and not to draw on parametric knowledge.
2. **Inline citation.** The model must cite sources as `[Source N]` inline, enabling the UI to show the corresponding retrieved chunks.
3. **Explicit uncertainty.** If the retrieved context does not contain enough information to answer, the model is instructed to say so explicitly rather than speculate. This directly improves the faithfulness score in evaluation.

Each retrieved chunk is formatted in the context block as:

```
[Source N | {League} Rulebook | {Section}]
{chunk_text}
```

or, for Layer 2 chunks:

```
[Source N | HoopStudent Basketball Encyclopedia | {Term} | {Section Type}]
{chunk_text}
```

One Gemma-specific issue was encountered: Gemma's chat template does not support the `system` role. The system prompt was therefore merged into the `user` turn.

4.6 Evaluation

The evaluation framework (`evaluate.py`) implements LLM-as-a-Judge scoring across two metrics, following the RAGAS-style methodology.

Faithfulness

Faithfulness measures the fraction of factual claims in the generated answer that are directly supported by the retrieved context — independent of the claim's factual truth in the real world. The pipeline proceeds in two steps:

Step 1 — Claim Extraction. The judge LLM is prompted to extract every atomic factual claim from the generated answer and return them as a JSON array. Atomic claims are short declarative sentences asserting a single fact; hedges, meta-statements, and opinions are excluded. The JSON output format makes parsing deterministic and avoids line-based fallbacks in the common case.

Step 2 — Claim Verification. Each extracted claim is presented to the judge alongside the full concatenated retrieved context. The judge returns a structured verdict in the format `VERDICT: SUPPORTED / NOT SUPPORTED` followed by a one-sentence reason. The regex parse targets the `VERDICT:` prefix specifically, making extraction robust to variation in the surrounding prose.

The faithfulness score is the fraction of supported claims:

$$\text{Faithfulness} = \frac{|\text{supported claims}|}{|\text{total claims}|}$$

Answer Relevancy

Relevancy measures how well the generated answer addresses the original query, using the following steps:

Step 1 — Question Generation. The judge LLM generates three distinct questions that the generated answer could plausibly be a response to.

Step 2 — Embedding Similarity. All three generated questions and the original query are embedded using `sentence-transformers/all-MiniLM-L6-v2`, a lightweight 80 MB model that runs efficiently on CPU. Since embeddings are L2-normalised, cosine similarity reduces to a dot product. The relevancy score is the mean cosine similarity between each generated question and the original query:

$$\text{Relevancy} = \frac{1}{3}\sum_{i=1}^{3} \cos(\mathbf{q}_{\text{orig}},\, \mathbf{q}_i)$$

The use of `all-MiniLM-L6-v2` for relevancy (rather than BGE-M3) is intentional: the similarity computation is a lightweight sentence-pair comparison for which a small bi-encoder is entirely adequate, and avoiding a second BGE-M3 load keeps memory pressure manageable on the free-tier deployment.

Judge Model and Reproducibility

The idea with the judge model was to use a smaller LLM whose only task would be for claim extraction, claim verification and question generation aspect of the evaluation.

All judge LLM calls use `temperature = 0.0` to make scores reproducible across evaluation runs. The judge client was initially configured to use OpenRouter (same as the generator), but was subsequently switched to Cohere's `tiny-aya` model due to OpenRouter's daily

request limits being insufficient for a 15-query evaluation set with multi-step per-query calls. The structured output format (`VERDICT:` prefix, JSON arrays) was designed to be parser-friendly regardless of which judge model is in use.

Test Set

A fixed set of 15 basketball queries is used for evaluation, spanning three categories: NBA-specific rules (defensive three-second violation, foul-out limits, out-of-bounds procedures, technical fouls, shot clock reset rules), FIBA-specific rules (shot clock duration, jump ball resolution, legal screens, backcourt violations, inbound timing), and tactical strategy (pick and roll mechanics, zone vs. man-to-man defense, box-and-one purpose, isolation plays, Princeton offense principles). Using a fixed test set ensures that ablation study results are directly comparable across configurations.

UI Integration

Evaluation results are streamed to the front-end via a Server-Sent Events (SSE) endpoint so that relevancy and faithfulness scores appear progressively as each query completes, rather than after the full 15-query batch. For faithfulness, the reasoning of the LLM for declaring the claim as Supported or Unsupported is also provided.

For automated evaluation of the test set, for the purposes of examination, there is a button on the top right of

NOTE: the UI works best on laptops/PCs, isn't made mobile friendly also note latency in the first prompt fed into the model

"Evaluate All"

5 Improvements to the First iteration

There were several challenges based on which some improvements were made, which led to V1 and V1.2 versions:

1. CHALLENGE OF EVALUATION LLM GIVING TOO MANY REQUESTS ERROR, which led to shifting to gemma3.1 4B judge model, with minor modifications in system prompt for hudge

2. The output generated was in a BIG BLOB/WALL OF TEXT WHICH IS NOT HIGHLY READABLE/UNDERSTANDABLE. This led to changing the system prompt, and modifying the UI

3. There was one issue for which there was no fix: a version mismatch of the transformers package between Docling and BGE(FlagMebedding) modules. Had to separate the pre-processing and embedding environments on Kaggle

   more context: After implementing the complete end to end pipeline, I found the following changes necessary

   first change the evaluation llm: changed to gemma3.1 4b

   then change the system prompt for both eval and generation generation: llm outputs a block of text, since I had the prompt of "be concise but complete" and strictly use sources, it's just stitching facts together mechanically without any formatting logic flow evaluation: we should allow for paraphrasing, too strict https://app.ailog.fr/en/blog/guides/prompt engineering-rag link used for seeing how to prompt used the instruction heirarchy point from here

   this is v1.1: v1.1 with the system prompt, UI improved, LLM change improvement make a note about how there are different versions/strategies, and then you can go to branch and see the code changes there, and the readme file will explain the differences

   NOTE: THAT I USED NAMESPACES IN PINECONE FOR TESTING THE DIFFERENT STRATEGIES. SINCE I FOUND OUT ABOUT IT AFTER COMPLETING THE FIRST IMPLEMENTATION, THE FIRST CHUNKING STRAT HAS "" DEFAULT NAMESPACE, MEANWHILE THE OTHER HAS IDK

   Note how added a 1 minute buffer because of api rate limits being reached for the automated evaluation test judge llm

This led to V1.2:

NOTE: THIS IS VERY IMPORTANT, MAYBE SIGNALLING WE NEED MMR!!!! also point that we found this issue using human evaluation, with only relevancy and faithfulness automated evaluation we wouldn't have been able to find this issue

Figure 5.1: Enter Caption

NOTE THIS did not get solved by MMR, even after experimenting with different lambda values for MMR:

Figure 5.2: Enter Caption

why didn't it? Single embedding for a comparative query: The entire query is encoded

into ONE dense vector and ONE sparse vector. The dense embedding will produce a single point in vector space that represents the "average" meaning of the query. Since the query mentions both defenses, but 1-2-1-1 appears more prominently (and likely has more chunks in the index since it's more written about), the dense vector will be pulled more toward 1-2-1-1. The RRF pool bottleneck: Even with MMR, the pipeline is:

Dense top-20 → likely 15-20 1-2-1-1 chunks Sparse top-20 → both terms present in sparse, but 1-2-1-1 likely dominates RRF merges → still dominated by 1-2-1-1 MMR$_{C}ANDIDATES =$ 15ßonlytop15fromRRFgointoMMRIf1−2−2chunksareranked16−20inRRF, theyneverevenreachMM

Why MMR didn't fully fix it: MMR got 2-1-2 instead of 1-2-2. This suggests 2-1-2 was just high enough in the RRF rankings to make it into the top-15 pool, but 1-2-2 was ranked even lower and didn't make the cut. Note with semantic chunking this issue was not there!

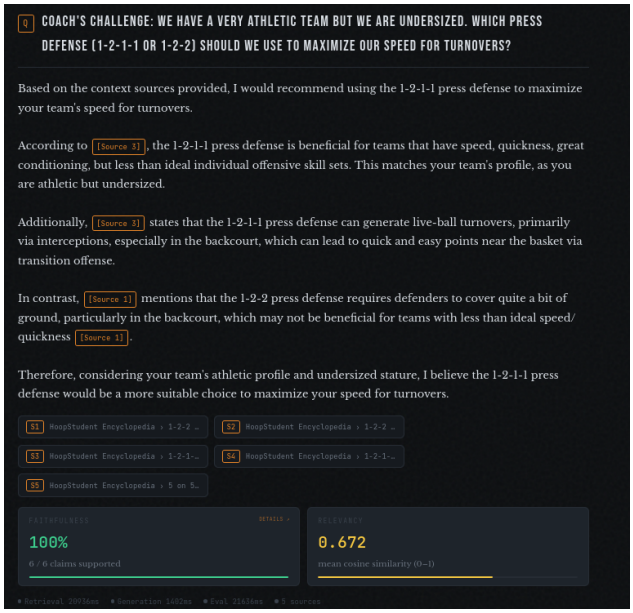

Figure 5.3: Enter Caption

so V1.2 concludes the work for this strategy.

5.1 Fixed size chunking

This involved using Docling in layer 1 for filtering text only (since we still need to be reliably picking text from the documents), but not used for final chunk boundaries, which are now fixed, with overlapping tokens

fixed size chunk size: 245 from layer 1, 3006 chunks from layer 2

note that querying fixed size chunks takes longer because it has full size chunks 512, which uses up a lot of the api rate limit, leading to delay.

Bad performance by fixed size chunking...

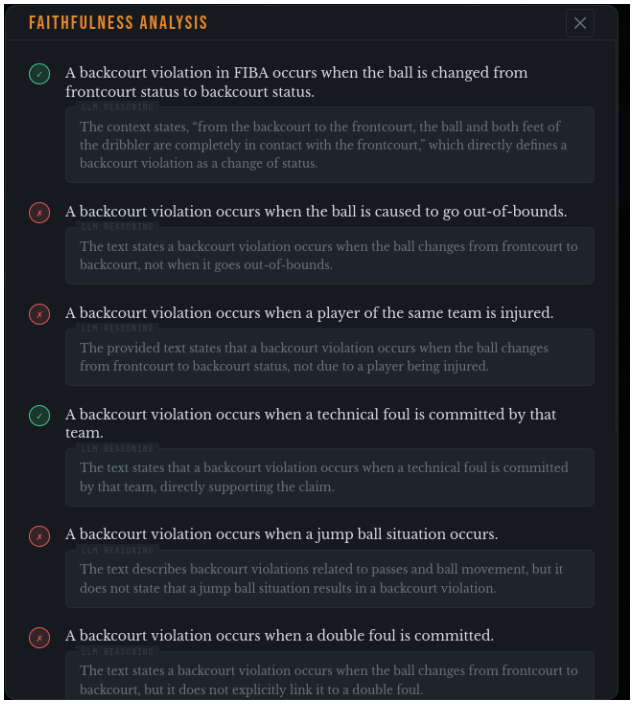

Figure 5.4: Enter Caption

5.2 Semantic Chunking

How semantic chunking works (the algorithm) The implementation follows the Greg Kamradt / LlamaIndex percentile-breakpoint approach:

Sentence tokenise the text block with NLTK's sent$_tokenize.Buildcontextwindows|eachwindowissentence$
$1, givingtheembedderapeekatwhatcomesnext.Embedeverywindowwithall-MiniLM-L6-$
$v2(alreadydownloadedbyevaluate.py, sonoextracost).Computecosinedistancebetweenconsecutivewindo$
$splitatsentenceboundariesasafallback.Chunksunder80charsaredroppedasnoise.$

The 85th percentile is configurable via BREAKPOINT_PERCENTILE. Raise it toward 95 for fewer, larger chunks; lower it toward 70 for many small ones.

total chunks: layer 1: 682 , layer 2: 5865 took the longest time out of all for chunking

5.3 Ablation Study

To compare each chunking and retrieval strategy, a test set of 10 queries was used for each, with their faithfulness and relevancy scores averaged. A RUN EVAL button on the top right of the interface runs the automated test evaluation.

used MMR as 0.7, because in my experimentation gave the best result, 0.6 was too random and for specific queries, and o0.9 meant no visible effect of MMR

note that for the last test, slasher wasn't explicitly a chunk of its own, so a difficult query